

ANG-SDK 开发使用规范

ANG-SDK-006

修订历史

版本号	时间	修定记录	修订部门
v1.1.0	2023/04/20	初版发布	软件部
v1.1.1	2023/05/19	1. 增加监听功能相关的接口和结构体 2. 修改 AS_SDK_SetStreamParam 接口和相关结构体 3. 增加获取相机型号接口 4. 增加网络接口和结构体 5. 增加设置时间戳接口 6. 增加获取分辨率能力集接口	软件部
v1.1.2	2023/06/08	示例代码通用化	软件部
v1.1.3	2023/06/21	修复获取 SN 号，优化 demo 逻辑	软件部
v1.1.4	2023/06/28	ROS/ROS2 demo 中点云消息增加时间戳和序号	软件部
v1.1.5	2023/07/03	修复网络设备遍历是可能出现的问题	软件部
v1.1.6	2023/07/06	hp60c/hp60cn/nuwa/vega 配置文件增加选择有序或无序点云的选项。	软件部
v1.1.7	2023/07/11	1. 增加 hp60v 深度校正 2. 更改 nuwa 配置文件部分选项命名	软件部
v1.1.8	2023/07/12	1. 优化 kunlun 算法 2. 示例代码增加判断是否虚拟机和是否匹配 nuwa 设备功能	软件部
v1.1.9	2023/07/14	1. 修复 NUWA hp60v(sh58)获取内外参错误问题	软件部
v1.1.10	2023/07/27	1. 增加 hp60c/hp60cn 模组在 ros/ros2 上支持预览彩色点云功能 2. 修复 ros2 程序退出时崩溃的问题 3. 修复 kunlun-a / kunlun-c / hp60c / hp60cn 固件升级失败问题 4. 修复 v4l2 模块关闭数据流时崩溃问题 5. 修复部分 nuwa 系列模组设置参数不生效问题 6. 更新增加 nuwa 配置文件选项	软件部
v1.1.11	2023/08/02	1、修复多模组情况下 hp60cn 深度图闪烁的问题	软件部
v1.2.0	2023/08/11	1. 增加特定相机的 launch 文件、mono8 话题发布、ros1 tf 树话题的发布； 2. 增加对 RGB 和深度对齐状态的获取 3. 修复已知问题，优化内部功能。	软件部
v1.2.1	2023/08/21	1. 增加 ros2 TF 树话题的发布； 2. 增加 X100 320x200 的分辨率； 3. 增加升级功能的使用说明； 4. 优化 x100 内部逻辑	软件部
v1.2.2	2023/08/22	1. 优化 x100 深度数据处理算法 2. 新增 x100 帧率配置。	软件部
v1.2.3	2023/08/29	1. 修复 kunlun 系列模组远近景切换卡顿问题 2. 增加 kunlun 系列模组 ros2 使用示例代码	软件部
v1.2.4	2023/08/31	1. 修复 nuwa 相机匹配多个节点可能会出现错误的问题	软件部

v1.2.5	2023/09/04	1、放宽网络相机热插拔检测时间从 2s->6s	软件部
v1.2.6	2023/09/07	新增 XB40、HP60、HP60V 帧率配置。	软件部
v1.2.7	2023/09/11	1. 修复导致深度去噪参数错误的 bug 2. 删除非热插拔 demo 中的自动重启功能 3. 修复网络相机监听机制的 bug	软件部
v1.2.8	2023/09/13	1. 优化日志打印，优化网络广播	软件部
v1.2.9	2023/10/23	1. 优化 NUWA 相机参数配置顺序逻辑 2. 增加 HP60CN 丢帧过滤检测	软件部
v1.2.10	2023/11/10	1. 添加 NUWA 各个型号的升级、能力集获取 2. 优化发布包结构和体积 3. 开发文档添加内参数据、SDK 调用限制说明、发布包使用说明 4. 删除非热插拔参考例子、原热插拔例子文件名去除 listener 关键字 5. 修复网络相机创建句柄偶发失败的问题 6. 修复 kunlun 系列 ros/ros2 话题发布异常	软件部
v1.2.11	2023/12/12	1. 修改 HP60 模组降噪参数(最大散斑大小 denoiseMaxSpeckleSize) 从原来 100 修改为 800 2. 修改 HP60C、HP60CN 相机内参处理 :cyRgb 值从源数据 (rawData[12]+6)/2.25, 修改为 (rawData[12]+6)/2.25 解析后赋值 3. 修改 HP60C、HP60CN 配置参数修改顺序: 从原开流后设置, 修改为开设备后到开流前设置 4. 修改 KunlunA、KUNLUNC 图像 raw 解析顺序, 从原左上角左往右、上往下逐行解析, 修改为从右下角右往左, 下往上逐行解析	软件部
v1.2.12	2023/12/28	1. KUNLUN-A/C 实现获取分辨率能力集接口 2. KUNLUN-A 修复获取固件版本号异常问题 3. KUNLUN-A/C 读取本地标定文件方式改为读取模组内的标定数据 4. KUNLUN-A/C 设置分辨率的宽高参数改为需要填入最终输出图像的宽高 5. 更新 KUNLUN 配置文件 6. 增加 WINDOWS 监听功能中枚举设备的功能	软件部
v1.2.13	2024/01/04	1. 修复 Hp60CN 内外参输出错误导致深度和彩色图无法进行对齐问题	软件部
v1.2.14	2024/01/29	1. 更新 KUNLUN-A/C 算法库 V3_20240116 2. 修复 KUNLUN-A/C windows 版本返回帧序号异常问题 3. 新增 ROS/ROS2 话题发布名称加上节点名称作为前缀 4. 新增 ROS/ROS2 话题订阅后相机开流并发布相应话题的功能 5. 修复 KUNLUN-A/C ROS/ROS2 CameraInfo 话题异常问题 6. 修改 ROS/ROS2 话题序号以区分多模组或多节点启动的话题名称 7. 修复 HP60CN 偶发打开设备失败 8. 修复 HP60CN 偶发崩溃问题 9. 优化 KUNLUN-A/C 出图时间 10. 增加 KUNLUN-A 默认内参	软件部
v1.2.15	2024/02/21	1. 新增 hp60 320x240 分辨率的支持 2. 修复 KUNLUN-A 固件 OTA 升级回调进度异常 3. 修复 KUNLUN 寄存器配置异常的问题	软件部

		4. 修改 KUNLUN 默认帧率由 20 帧改为 10 帧 5. 修复 ROS/ROS2 多模组或多节点启动序号异常 6. 新增 ROS1 nodelet 功能 7. 新增 HP60C/HP60CN/KUNLUN-C jpeg 格式的数据输出到回调, 并在 linux-demo 显示	
v1.2.16	2024/02/23	1. 修复 KUNLUN-A/C 开流后寄存器设置不生效导致深度图闪烁的问题 2. 更新 KUNLUN-A/C 算法库 V3_20240223 修复自动积分函数的问题	软件部
v1.2.17	2024/03/25	1. 优化 HP60C 增益设置方式	软件部
v1.2.18	2024/03/28	1. 镜像 HP60CN 深度图和彩色图	软件部
v1.2.19	2024/04/03	1. 更新 KUNLUN 算法库 2. 修复 linux demo 预览图像异常问题	软件部
v1.2.20	2024/04/15	1. 更新 KUNLUN 算法库	软件部
v1.2.21	2024/04/25	1. 优化 NUWA 系列模组的固件升级顺序	软件部
v1.2.22	2024/05/16	1. 增加获取相机私有数据的空间大小接口 2. 增加向相机写入私有数据的接口 3. 增加从相机读取私有数据的接口 4. 更新 KUNLUN-A/C 配置文件	软件部
v1.2.23	2024/07/18	1. 增加 VEGA、CHANGJIANG-B、TAISHAN、TANGGULA-A 型号模组 2. 增加 VEGA 型号深度保留小数输出功能, 默认关闭, 可在配置文件中修改 Decimal 为 true 开启 3. 修复 ros2 组件链接错误 4. 修复网络相机偶发关流崩溃问题 5. 修复网络相机多相机工作互相干扰导致打开设备失败的 bug 6. 镜像 HP60CN 深度和 RGB 方向	软件部
v1.2.24	2024/08/08	1. 修复 OTA 包内多个文件升级失败的问题 2. 修复 changjiangB 模组出红外图返回宽高不正确的问题	软件部
v1.2.25	2024/08/28	1. 修复 AS_SDK_DestroyCamHandle 访问已释放内存问题 2. 修复 Linux/ros/ros2 的 demo 退出未释放内存问题 3. TANGGULA-A 增加从模组读取图像分辨率、从配置文件读取深度移位位数功能 4. 增加 02 camera (仅支持 WINDOWS) 5. 增加 TANGGULA-B 模组 6. 更新 changjiangB 的深度拼接算法 7. 增加 OTA 升级文件类型 warp 8. 修复 changjiangB 模组开关设备内存泄漏问题	软件部
v1.2.26	2024/09/03	1. 更新 TANGGULA-A 配置文件 2. TANGGULA-B 增加彩色图像和 IR (散斑) 图像同时输出功能	软件部
v1.2.27	2024/09/12	1. TANGGULA-A 和 TANGGULA-B 默认使用最大帧率 2. 修复 hp60c/hp60cn 缩小分辨率后点云畸变问题	软件部
V1.2.28	2024/10/15	1. 更新 TANGGULA-B 配置文件, 默认开启对齐 2. TANGGULA-A/TANGGULA-B 增加默认内外参 3. 增加对 XB40-V3 串口通信版本固件的支持	软件部
V1.2.28	2024/10/21	1. 修正 ANG-LINUX-ROS 使用说明文档 9.5 的错误描述	软件部

Angstrong Technology

The copyright of this manual belongs to Shenzhen Angstrong Technology Co., Ltd. No part of this manual may be translated into other languages or reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying and recording. Information in this document is subject to change without notice and does not represent a commitment on the part of Shenzhen Angstrong Technology Co., Ltd. Please contact us for the latest product information. The manual is only for customers who have purchased the product.

本手册版权归深圳市安思疆科技有限公司所有，未经许可，任何单位和个人都不得以电子的、机械的、磁性的、光学的、手工的等形式复制、传播、转录和保存该出版物，或翻译成其他语言版本。一经发现，将追究其法律责任。深圳市安思疆科技有限公司保留更改本说明书的权利，届时恕不另行通知，请谅解。请在订购时联系我们以获得产品最新信息。本使用说明仅面向已购买产品的顾客。在编写说明内容时，仅针对产品使用者。本产品说明书可能无法满足非产品购买者的疑问，请予以谅解。

目录

1. 简介 10

1.1 产品概述 10

1.2 环境要求 10

1.2.1 运行环境 10

1.2.2 系统要求 10

1.3 SDK 功能介绍 10

1.3.1 获取深度 10

1.3.2 获取点云 10

1.3.3 获取 IR 11

1.3.4 获取 RGB 11

1.3.5 获取 PEAK 11

2.SDK 集成指南 12

2.1 包结构说明 12

2.2 示例程序运行说明 13

2.2.1 相机配置文件说明 13

2.2.2 运行 Ubuntu (Linux) 系统示例程序 13

2.2.3 运行 ROS/ROS2 系统示例程序 13

2.2.4 如何交叉编译 13

2.2.5 运行 Windows 系统示例程序 14

2.3.1 主动获取相机列表 15

2.3.2 基于热插拔的相机管理 16

3. 重要数据结构说明 17

3.1 流接口数据类型 17

3.1.1 AS_CAM_PTR 17

3.1.2 AS_FRAME_Type_e 17

3.1.3 AS_Frame_s 17

3.1.4 AS_SDK_Data_s 18

3.1.5 AS_SDK_MERGE_s 19

3.1.6 AS_CAM_StreamCallback 19

3.1.7 AS_CAM_Stream_Cb_s 19

3.1.8 AS_CAM_MergeFrameCallback 20

3.1.9 AS_CAM_Merge_Cb_s 20

3.1.10 AS_Listener_Callback 20

3.1.11 AS_LISTENER_CALLBACK_S 21

3.1.12 AS_LISTENER_TYPE_E 21

3.1.13 IMG_TYPE_FLG 21

3.1.14 AS_POINTCLOUD_s 22

3.1.15 AS_CONFIG_INFO_s 22

3.2 通信接口数据类型	23
3.2.1 AS_CAM_Parameter_s	23
3.2.2 AS_STREAM_Param_s	24
3.2.3 AS_MEDIA_TYPE_E	25
3.2.4 AS_CAM_UpGradeCallback	25
3.2.5 AS_CAM_Upgrade_Dev_s	26
3.2.6 AS_TOFMODE_E	26
3.2.7 AS_SDK_CAM_MODEL_E	26
3.2.8 AS_CAM_ATTR_TYPE_E	28
3.2.9 AS_USB_DEV_ATTR_S	28
3.2.10 AS_NETWORK_DEV_ATTR_S	29
3.2.11 AS_USB_WIN_DEV_ATTR_S	29
3.2.12 AS_CAM_ATTR_S	30
3.2.13 AS_NET_DEV_Attrs_s	30
3.2.14 AS_TIME_STAMP_TYPE_E	30
4.ANG SDK 核心 API 说明	32
4.1 核心接口定义	32
4.1.1 初始化 SDK	32
4.1.2 去初始化 SDK	32
4.1.3 获取相机列表	32
4.1.4 释放相机列表	33
4.1.5 打开相机	33
4.1.6 关闭相机	34
4.1.7 注册图像数据回调函数	34
4.1.8 注册图像融合数据回调函数	35
4.1.9 打开相机数据流	36
4.1.10 关闭相机数据流	36
4.1.11 启动设备监听功能	37
4.1.12 停止设备监听功能	37
4.1.13 创建相机句柄	37
4.1.14 销毁相机句柄	38
4.1.15 获取相机属性	38
4.1.16 获取 SDK 版本号	39
4.2 通信接口定义	39
4.2.1 获取相机固件版本号	39
4.2.2 获取流的类型	40
4.2.3 获取相机 SN 序列号	41
4.2.4 升级相机固件	41
4.2.5 获取相机内外参	42
4.2.6 获取相机固件的模式	43
4.2.7 获取分辨率能力集	43
4.2.8 设置图像参数	44
4.2.9 获取图像参数	44
4.2.10 获取相机型号	45
4.2.11 设置相机网络信息	45

- 4.2.12 获取相机网络信息 46
 - 4.2.13 设置时间戳类型 46
 - 4.2.14 获取相机配置信息 47
 - 4.2.15 获取相机私有数据的空间大小 48
 - 4.2.16 向相机写入私有数据 48
 - 4.2.17 从相机读取私有数据 49
- 5. 帮助 49
 - 5.1 注意事项 49
 - 5.2 FAQ 49
 - 5.2.1 如何不以 root 权限运行 linux 程序 49
 - 5.2.2 如何不以 root 权限运行 ROS 节点 50
 - 5.2.3 执行 SDK 提供的脚本报错 50
 - 5.2.4 在 ubuntu20.04/ubuntu22.04 ROS 上编译报错 51
 - 5.2.5 NUWA 系列相机在虚拟机中匹配异常问题 51
 - 5.2.6 KUNLUN-A 相机在虚拟机中运行异常 52
 - 5.2.7 多相机运行出流异常问题 52
 - 5.2.8 在 windows10/windows11 打开 USB 相机失败 52

1.简介

1.1 产品概述

安思疆 SDK API 为第三方应用集成摄像头模组功能提供了一站式服务。基于 SDK 可快速为客户提供二次开发服务,防止在使用 Angstrong SDK API 的过程中由于不规范的调用而引起其他问题。

SDK 用于驱动安思疆科技各系列相机，提供实时获取深度图像、点云图、IR 图、RGB 图和 PEAK 图等功能。

1.2 环境要求

1.2.1 运行环境

ARM/AARCH64、X86_64 架构。

1.2.2 系统要求

Linux: Ubuntu16.04/Ubuntu18.04/Ubuntu20.04/Ubuntu22.04（推荐使用 Ubuntu18.04）

ROS1: Kinetic Kame/Melodic Morenia/Noetic Ninjemys（推荐使用 Melodic Morenia）

ROS2: Foxy Fitzroy/Galactic Geochelone/Humble Hawksbill（推荐使用 Humble Hawksbill）

Windows.

1.3 SDK 功能介绍

1.3.1 获取深度

支持驱动安思疆相机，实时获取深度图；

深度数据的空间格式如下

表 1 深度图格式

位宽度	16
单位	mm
类型	unsigned short

1.3.2 获取点云

支持驱动安思疆相机，实时获取点云图；

在图像数据回调中得到点云图。

1.3.3 获取 IR

支持驱动安思疆相机，实时获取 IR 图；

在图像数据回调中得到 IR 图。

1.3.4 获取 RGB

支持驱动安思疆相机，实时获取 RGB 图；

在图像数据回调中得到 RGB 图。

1.3.5 获取 PEAK

支持驱动安思疆相机，实时获取 PEAK 图；

在图像数据回调中得到 PEAK 图。

2.SDK 集成指南

2.1 包结构说明

版本变更说明：从 v1.2.10 开始之后的版本，所提供的 linux/ros/ros2 的 demo 均是热插拔机制实现的参考例子，等同于此前版本带 xxx_listener 的参考例子，此前版本的非热插拔例子不再提供。切换新版本后，请注意 ros 系统话题发布名称的变动。

从 v1.2.10 开始之后的版本,发布包为 xxx.tar.xz 的格式。在 Linux 环境下,使用 `tar -xJvf` (注意是大写的 J) 解压;在 windows 环境下,可用解压工具解压。Linux 环境下解压后执行以下操作得到完整 SDK 包:

```
cd ${releasePackage}/demo/  
./unpack linux ros.sh
```

结构说明如下:

AngstrongCameraSdk vx. x. x. 20xxxxxx

└── CHANGELOG	SDK 更新记录
└── docs	SDK/ROS 开发使用说明和其他信息表
└── demo	
└── linux_ros	
└── linux	linux 系统下简单调用的参考例子
└── configurationfiles	配置文件
└── include	demo 头文件
└── libs	
└── include	SDK 头文件
└── lib	SDK 库文件
└── aarch64-linux-gnu	aarch64-linux-gnu-g++版本
└── arm-linux-gnueabi	arm-linux-gnueabi-g++版本
└── x86_64-linux-gnu	x86_64-linux-gnu-g++版本
└── src	demo 源文件
└── ros	ros 系统下的 ROS1 参考例子
└── ros2	ros 系统下的 ROS2 参考例子
└── windows	windows 系统的参考例子

2.2 示例程序运行说明

2.2.1 相机配置文件说明

相机配置文件在目录 `configurationfiles` 下，调用“[打开相机](#)”接口时需要将配置文件路径传进去。默认提供的都是加密的配置文件，如果用户需要修改配置文件参数，请联系 FAE 获取。

2.2.2 运行 Ubuntu(Linux) 系统示例程序

进入到 `demo/linux_ros/linux` 目录，其中 `build.sh` 是编译脚本，`run_ascamera.sh` 是运行示例程序的脚本。首先，执行 `build.sh` 进行编译，然后执行 `run_ascamera.sh` 脚本运行程序即可。执行程序需要 root 权限，脚本中会以 `sudo` 方式运行

编译：

```
./build.sh
```

运行：

```
./run_ascamera.sh
```

2.2.3 运行 ROS/ROS2 系统示例程序

进入到 `demo/linux_ros/ros` 或 `ros2` 目录，其中 `build.sh` 是编译脚本，`run_ascamera.sh` 是启动节点的脚本。首先，执行 `build.sh` 进行编译，然后执行 `run_ascamera.sh` 脚本运行程序即可。启动节点需要 root 权限，脚本中会以 `sudo` 方式运行

编译：

```
./build.sh
```

启动节点：

```
./run_ascamera.sh
```

2.2.4 如何交叉编译

交叉编译，是在一个平台上生成另一个平台上的可执行代码。如在 `x86_64` 架构的 Linux 系统上编译出可在 `arm/aarch64` 架构的 Linux 系统上运行的可执行文件。安思疆 NUWA Linux SDK 的 demo 代码中，会根据宿主机的体系架构自动链接对应的库文件，所以将 SDK 放到支持编译器的 arm 开发板中，编译方法与在 `x86_64` 架构相同，只需执行目录下的 `build.sh` 脚本即可。如需在 `x86_64` 的 Linux 中交叉编译生成 `arm/aarch64` 的可执行文件，请参考以下章节。目前 SDK 中已编译好的 `arm/aarch64` 版本的动态链接库，可以在 `/libs/lib` 目录查看，

对应的目录名为编译器的名称。如没有所需要的 arm/aarch64 版本的动态链接库, 则需提供 x86_64 上 Linux 的交叉编译工具链重新编译生成相关的动态链接库。

2.2.5 运行 Windows 系统示例程序

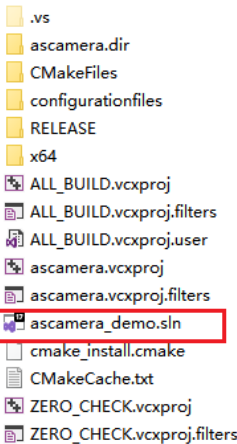
2.2.5.1 直接运行已编译好的程序

进入到 demo/windows/release 目录, 将 demo/windows/libs/lib/x86_64-win-msvc/ 目录下所有 dll 库拷贝到此处后, 双击执行文件即可。

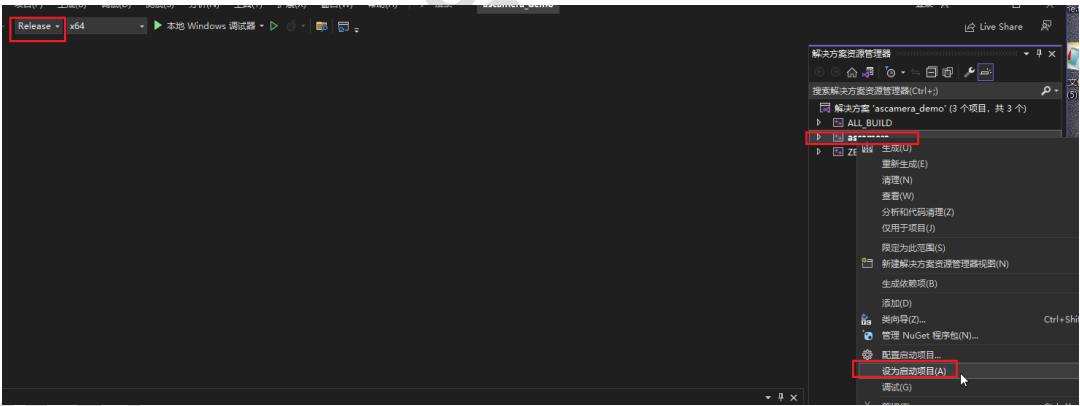
2.2.5.2 编译运行程序

步骤 1. 在 demo/windows 目录下新建构建目录 build, 使用命令 `cmake ..` 编译。

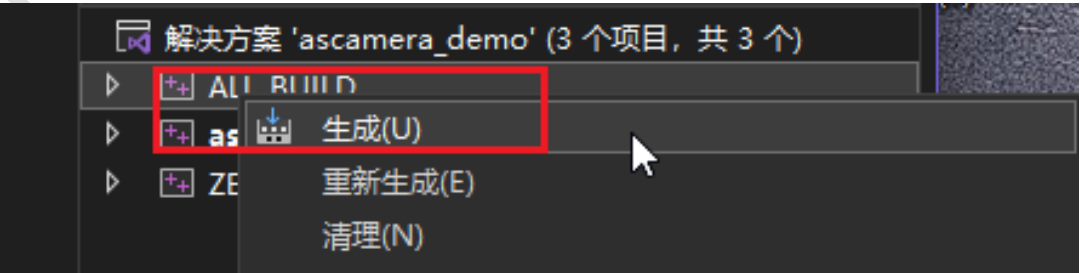
步骤 2. 双击 sln 后缀的文件。



步骤 3. 选择 Release 后, 右击右方的 ascamera 设为启动项目。



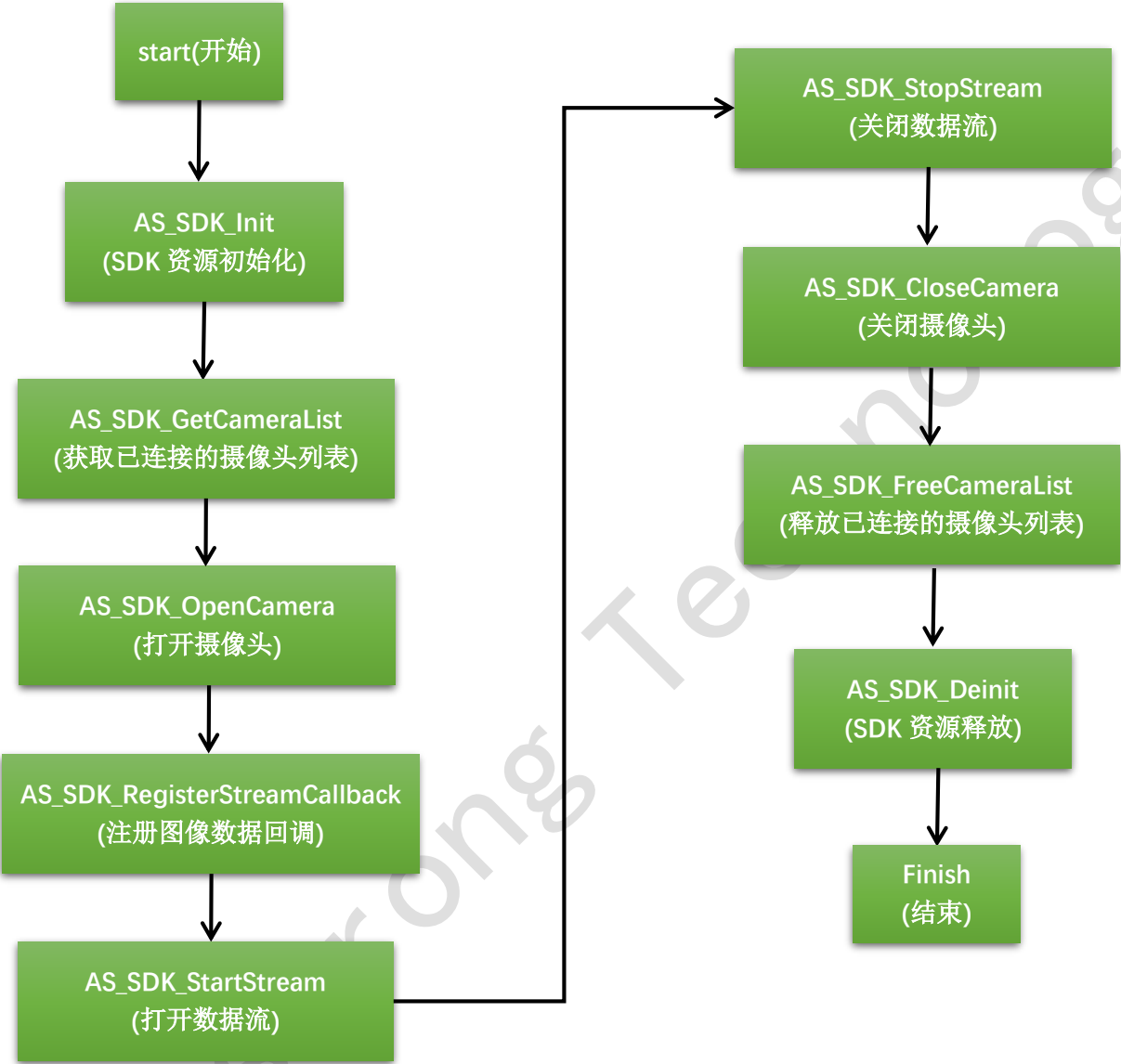
步骤 4. 右击右方的 ALL_BUILD, 点击生成。



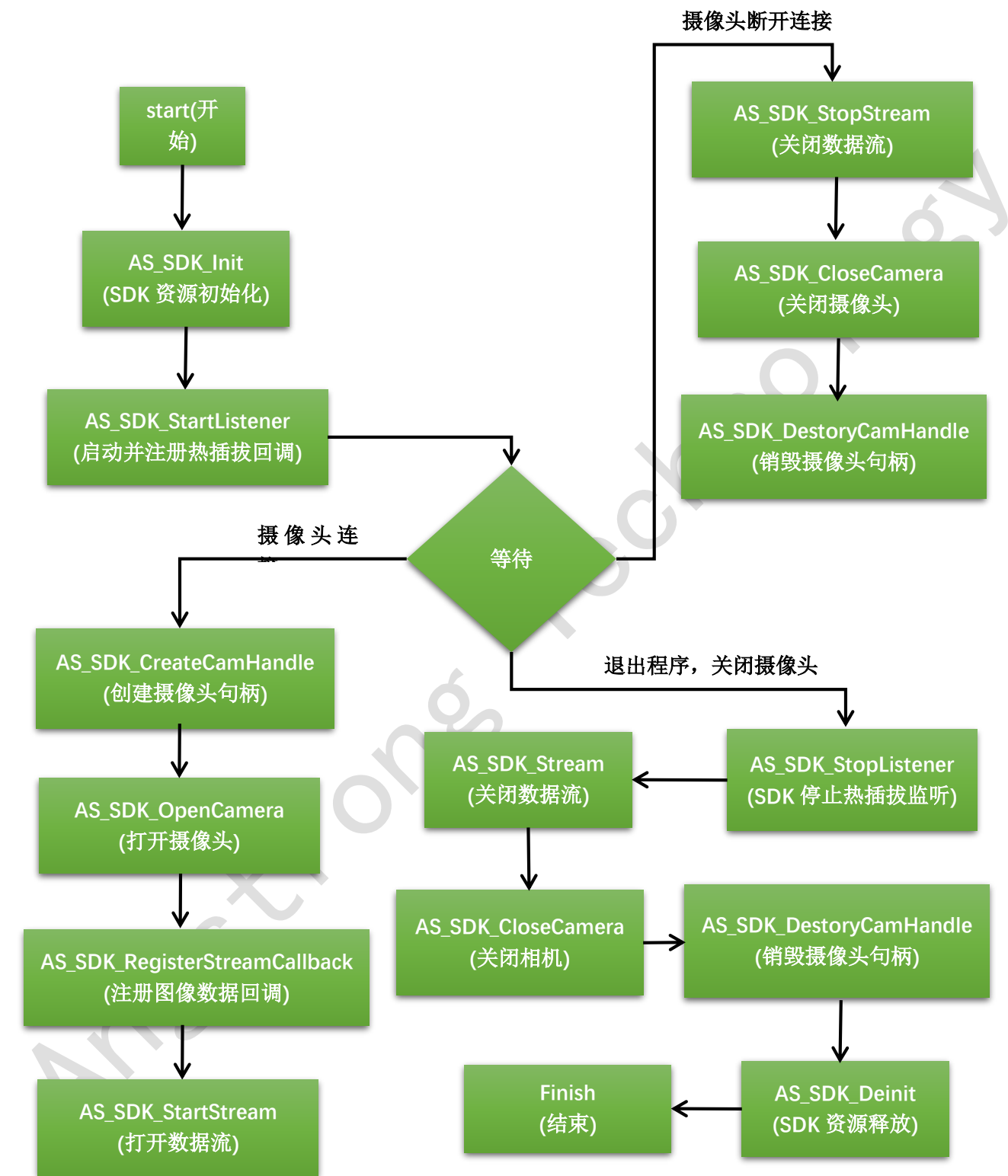
步骤 5. 进入 demo/win/build/RELEASE 目录, 双击 ascamera.exe 运行程序。

2.3 调用流程图

2.3.1 主动获取相机列表



2.3.2 基于热插拔的相机管理



3. 重要数据结构说明

3.1 流接口数据类型

3.1.1 AS_CAM_PTR

【说明】
相机句柄

【定义】
typedef void * AS_CAM_PTR;

3.1.2 AS_FRAME_Type_e

【说明】
数据帧类型

【定义】
typedef enum FRAME_TYPE_E {
AS_FRAME_TYPE_DEPTH = 0,
AS_FRAME_TYPE_RGB,
AS_FRAME_TYPE_IR,
AS_FRMAE_TYPE_POINTCLOUD,
AS_FRAME_TYPE_YUYV,
AS_FRAME_TYPE_PEAK,
AS_FRAME_TYPE_MJPEG,
AS_FRAME_TYPE_BUTT
} AS_FRAME_Type_e;

【参数】

Member	Description
AS_FRAME_TYPE_DEPTH	深度数据帧类型
AS_FRAME_TYPE_RGB	彩色图像帧类型
AS_FRAME_TYPE_IR	IR 图像帧类型
AS_FRMAE_TYPE_POINTCLOUD	点云数据帧类型
AS_FRMAE_TYPE_YUYV	YUYV 图像帧类型
AS_FRAME_TYPE_PEAK	PEAK 图像帧类型
AS_FRAME_TYPE_MJPEG	MJPEG 图像帧类型
AS_FRAME_TYPE_BUTT	无效值

3.1.3 AS_Frame_s

【说明】
帧结构

【定义】
typedef struct CAM_FRAME {

```
AS_FRAME_Type_e type;
unsigned int width;
unsigned int height;
void *data;
unsigned int size;
unsigned int bufferSize;
unsigned int frameId;
unsigned long long ts;
} AS_Frame_s;
```

【参数】

Member	Description
type	数据帧类型
width	图像宽，单位为像素
height	图像高，单位为像素
data	数据缓存地址
size	有效数据的大小，单位为字节(Byte)
bufferSize	数据缓存的空间总大小，单位为字节(Byte)
frameId	该帧的序号
ts	该帧的时间戳，取自(调用 sdk 所运行的)系统的系统时间，单位为毫秒

3.1.4 AS_SDK_Data_s

【说明】

图像数据结构。当使用的相机型号不存在对应的图像数据或未开启对应的数据流的时候，输出的图像数据长度 xxxImg.size 为 0。注意，当 RGB 相机开启 RGB 图像和深度图像时，若设置二者图像对齐，点云数据 pointCloud 所对应的深度图像是对齐的深度图像，反之则为未对齐的深度图像。配置 RGB 图像和深度图像是否对齐，可通过替换对应的配置文件，请联系 FAE 获取方式。

【定义】

```
typedef struct CAM_DATA {
    AS_Frame_s depthImg;
    AS_Frame_s rgbImg;
    AS_Frame_s irImg;
    AS_Frame_s pointCloud;
    AS_Frame_s yuyvImg;
    AS_Frame_s peakImg;
    AS_Frame_s mjpegImg;
    std::vector<AS_POINTCLOUD_s> pointCloud2;
} AS_SDK_Data_s;
```

【参数】

Member	Description
depthImg	深度数据, unsigned short 类型，单位为毫米
rgbImg	彩色图像数据，BGR 格式，unsigned char 类型
irImg	红外图像数据，unsigned char 类型

Member	Description
pointCloud	深度图像对应的点云数据, float 类型, 按照 xyz 顺序排列, z 为深度(距离)值, 单位毫米。无序点云已丢弃了无效点, 有序点云未丢弃无效点。
yuyvImg	YUYV 数据, unsigned char 类型
peakImg	PEAK 图像数据, unsigned char 类型
pointCloud2	极坐标点云数据(仅支持 KONDYOR 使用, 其他类型模组请忽略)
mjpegImg	MJPEG 图像帧类型, 仅支持 RGB 相机使用, 其中 HP60C/HP60CN 需要做沿 x 轴翻转(上下翻转)操作

【说明】

KUNLUN-A 与 KUNLUN-C 相机的 pointCloud 数据为有序点云

3.1.5 AS_SDK_MERGE_s

【说明】

融合图像数据结构

【定义】

```
typedef struct CAM_MERGE {
    AS_Frame_s depthImg;
    AS_Frame_s pointCloud;
} AS_SDK_MERGE_s;
```

【参数】

Member	Description
depthImg	深度数据, unsigned short 类型, 单位为毫米
pointCloud	深度图像对应的点云数据, float 类型, 按照 xyz 顺序排列, z 为深度(距离)值, 单位毫米。无序点云已丢弃了无效点, 有序点云未丢弃无效点。

3.1.6 AS_CAM_StreamCallback

【说明】

图像数据回调函数

【定义】

```
typedef void(*AS_CAM_StreamCallback)(AS_CAM_PTR pCamera, const AS_SDK_Data_s *pstData, void *privateData);
```

【参数】

Member	Description
pCamera	相机句柄
pstData	图像数据
privateData	用户私有数据

3.1.7 AS_CAM_Stream_Cb_s

【说明】

图像数据回调数据结构

【定义】

```
typedef struct STREAM_CALLBACK_S {
    AS_CAM_StreamCallback callback;
    void *privateData;
} AS_CAM_Stream_Cb_s;
```

【参数】

Member	Description
callback	数据回调函数
privateData	用户私有数据

3.1.8 AS_CAM_MergeFrameCallback

【说明】

融合图像数据回调函数

【定义】

```
typedef void(*AS_CAM_MergeFrameCallback)(AS_CAM_PTR pCamera, const AS_SDK_Merge_s *pstData, void *privateData);
```

【参数】

Member	Description
pCamera	相机句柄
pstData	图像数据
privateData	用户私有数据

3.1.9 AS_CAM_Merge_Cb_s

【说明】

融合图像数据回调数据结构

【定义】

```
typedef struct MERGE_CALLBACK_S {
    AS_CAM_MergeFrameCallback callback;
    void *privateData;
} AS_CAM_Merge_Cb_s;
```

【参数】

Member	Description
callback	数据回调函数
privateData	用户私有数据

3.1.10 AS_Listener_Callback

【说明】

监听功能回调函数

【定义】

```
typedef void(*AS_Listener_Callback)(AS_CAM_ATTR_S *attr, void *privateData);
```

【参数】

Member	Description
attr	相机信息
privateData	用户私有数据

3.1.11 AS_LISTENER_CALLBACK_S

【说明】

监听功能回调数据结构

【定义】

```
typedef struct AS_LISTENER_CALLBACK {
    AS_Listener_Callback onAttached;
    AS_Listener_Callback onDetached;
    void *privateData;
} AS_LISTENER_CALLBACK_S;
```

【参数】

Member	Description
onAttached	相机连接回调
onDetached	相机断开连接回调
privateData	用户私有数据

3.1.12 AS_LISTENER_TYPE_E

【说明】

监听类型，当使用 USB 相机时候，只需注册 USB 设备监听类型，反之只需注册网络监听类型。为降低 CPU 占用等情况，建议只注册需要的监听类型。

【定义】

```
typedef enum AS_LISTENER_TYPE {
    AS_LISTENNER_TYPE_USB = 0,
    AS_LISTENNER_TYPE_NET,
    AS_LISTENNER_TYPE_BUTT
} AS_LISTENER_TYPE_E;
```

【参数】

Member	Description
AS_LISTENNER_TYPE_USB	USB 类型
AS_LISTENNER_TYPE_NET	网络类型
AS_LISTENNER_TYPE_BUTT	无效值

3.1.13 IMG_TYPE_FLG

【说明】

图像类型标志

【定义】

```
#define DEFAULT_IMG_FLG    0x00000000
#define DEPTH_IMG_FLG      0x00000001
```

```
#define RGB_IMG_FLG          0x00000002
#define IR_IMG_FLG          0x00000004
#define POINTCLOUD_IMG_FLG  0x00000008
#define YUYV_IMG_FLG        0x00000010
#define PEAK_IMG_FLG        0x00000020
#define SUB_DEPTH_IMG_FLG   0x00000040
#define MJPEG_IMG_FLG       0x00000080
```

【参数】

Member	Description
DEFAULT_IMG_FLG	默认图像类型
DEPTH_IMG_FLG	深度图像类型
RGB_IMG_FLG	彩色图像类型
IR_IMG_FLG	IR 图像类型
POINTCLOUD_IMG_FLG	点云图像类型
YUYV_IMG_FLG	yuyv 图像类型
PEAK_IMG_FLG	PEAK 图像类型
SUB_DEPTH_IMG_FLG	附加深度图像类型（仅 CHANGJIANG-B 使用）
MJPEG_IMG_FLG	MJPEG 图像类型

3.1.14 AS_POINTCLOUD_s

【说明】

极坐标点云信息

【定义】

```
typedef struct AS_POINTCLOUD {
    double angle;
    double distance;
    double quality;
} AS_POINTCLOUD_s;
```

【参数】

Member	Description
angle	极坐标点云角度
distance	极坐标点云距离
quality	极坐标点云质量

【注】此结构体仅支持 KONDYOR 使用，其他类型模组请忽略。

3.1.15 AS_CONFIG_INFO_s

【说明】

彩色 RGB 相机配置信息

【定义】

```
typedef struct AS_CONFIG_INFO {
    bool is_Registration;
    bool is_pclOrganized;
} AS_CONFIG_INFO_S;
```

【参数】

Member	Description
is_Registration	深度数据和 RGB 数据是否对齐
is_pclOrganized	输出的是否为有序点云

【注】此结构体供查询 RGBD 相机配置信息使用，其他类型模组请忽略。

3.2 通信接口数据类型

3.2.1 AS_CAM_Parameter_s

【说明】

相机内外参。特别注意，深度相机和红外（IR）相机使用同一套参数，在默认配置的 RGB 相机中，深度图像和 RGB 图像一般是对齐输出的。若输出的是深度图像和 RGB 图像对齐的数据，红外/深度相机的焦距和光心坐标参数请使用 RGB 相机的焦距和光心坐标，反之使用原参数。

【定义】

```
typedef struct CAM_Parameter {  
    float fxir;  
    float fyir;  
    float cxir;  
    float cyir;  
    float fxrgb;  
    float fyrgb;  
    float cxrgb;  
    float cyrgb;  
  
    float R00;  
    float R01;  
    float R02;  
    float R10;  
    float R11;  
    float R12;  
    float R20;  
    float R21;  
    float R22;  
    float T1;  
    float T2;  
    float T3;  
  
    float K1ir;  
    float K2ir;  
    float K3ir;  
    float P1ir;  
    float P2ir;
```

```
float K1rgb;  
float K2rgb;  
float K3rgb;  
float P1rgb;  
float P2rgb;  
} AS_CAM_Parameter_s;
```

【参数】

Member	Description
fxir	红外/深度相机横向焦距
fyir	红外/深度相机纵向焦距
cxir	红外/深度相机光心横坐标
cyir	红外/深度相机光心纵坐标
fxrgb	彩色相机横向焦距
fyrgb	彩色相机纵向焦距
cxrgb	彩色相机光心横坐标
cyrgb	彩色相机光心纵坐标
R00	红外/深度与彩色相机外参旋转矩阵元素
R01	红外/深度与彩色相机外参旋转矩阵元素
R02	红外/深度与彩色相机外参旋转矩阵元素
R11	红外/深度与彩色相机外参旋转矩阵元素
R12	红外/深度与彩色相机外参旋转矩阵元素
R20	红外/深度与彩色相机外参旋转矩阵元素
R21	红外/深度与彩色相机外参旋转矩阵元素
R22	红外/深度与彩色相机外参旋转矩阵元素
T1	红外/深度与彩色相机外参平移矩阵元素
T2	红外/深度与彩色相机外参平移矩阵元素
T3	红外/深度与彩色相机外参平移矩阵元素
K1ir	IR 畸变参数
K2ir	IR 畸变参数
K3ir	IR 畸变参数
P1ir	IR 畸变参数
P2ir	IR 畸变参数
K1rgb	RGB 畸变参数
K2rgb	RGB 畸变参数
K3rgb	RGB 畸变参数
P1rgb	RGB 畸变参数
P2rgb	RGB 畸变参数

3.2.2 AS_STREAM_Param_s

【说明】

数据流的参数，包括分辨率和帧率。

【定义】


```
typedef struct STREAM_PARAM_S {
    int width;
    int height;
    int fps;
} AS_STREAM_Param_s;
```

【参数】

Member	Description
width	图像宽
height	图像高
fps	帧率

3.2.3 AS_MEDIA_TYPE_E

【说明】

流类型

【定义】

```
typedef enum MEDIA_TYPE_E {
    AS_MEDIA_TYPE_DEPTH,
    AS_MEDIA_TYPE_RGB,
    AS_MEDIA_TYPE_IR,
    AS_MEDIA_TYPE_PEAK,
    AS_MEDIA_TYPE_BUTT
} AS_MEDIA_TYPE_E;
```

【参数】

Member	Description
AS_MEDIA_TYPE_DEPTH	深度类型
AS_MEDIA_TYPE_RGB	RGB 类型
AS_MEDIA_TYPE_IR	IR 类型
AS_MEDIA_TYPE_PEAK	PEAK 类型
AS_MEDIA_TYPE_BUTT	无效值

3.2.4 AS_CAM_UpGradeCalllback

【说明】

固件升级状态回调函数

【定义】

```
typedef void(AS_CAM_UpGradeCallback)(AS_CAM_PTR pCamera, void *privateData,float fProcess);
```

【参数】

Member	Description
pCamera	由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。

Member	Description
privateData	用户私有数据
fProcess	升级进度，由底层输出，数值范围是 0.00-100.00

3.2.5 AS_CAM_Upgrade_Dev_s

【说明】

设备固件升级状态回调数据结构

【定义】

```
typedef struct CAM_UPGRADE_DEV_S {
    AS_CAM_UpGradeCallback_callback;
    void *privateData;
} AS_CAM_Upgrade_Dev_s;
```

【参数】

Member	Description
callback	设备固件升级状态回调函数
privateData	用户私有数据

3.2.6 AS_TOFMODE_E

【说明】

固件模式，非 ToF 相机请忽略，仅适用于 KUNLUN-A 相机

【定义】

```
typedef enum TOFMode {
    odd_mode = 0,
    even_mode = 1,
    merge_mode = 2,
    mode_but
} AS_TOFMODE_E;
```

【参数】

Member	Description
odd_mode	单独奇帧模式
even_mode	单独偶帧模式
merge_mode	奇偶交替帧模式
mode_but	无效值

3.2.7 AS_SDK_CAM_MODEL_E

【说明】

相机型号

【定义】

```
typedef enum CAMERA_MODEL_E {
    AS_SDK_CAM_MODEL_UNKNOWN,
    AS_SDK_CAM_MODEL_NUWA_XB40,
    AS_SDK_CAM_MODEL_NUWA_X100,
    AS_SDK_CAM_MODEL_NUWA_HP60,
    AS_SDK_CAM_MODEL_NUWA_HP60V,
    AS_SDK_CAM_MODEL_KUNLUN_A,
    AS_SDK_CAM_MODEL_KUNLUN_C,
    AS_SDK_CAM_MODEL_KONDYOR,
    AS_SDK_CAM_MODEL_KONDYOR_NET,
    AS_SDK_CAM_MODEL_HP60C,
    AS_SDK_CAM_MODEL_HP60CN,
    AS_SDK_CAM_MODEL_VEGA,
    AS_SDK_CAM_MODEL_CHANGA,
    AS_SDK_CAM_MODEL_CHANGJIANG_B,
    AS_SDK_CAM_MODEL_TANGGULA,
    AS_SDK_CAM_MODEL_TANGGULA_A,
    AS_SDK_CAM_MODEL_TAISHAN,
    AS_SDK_CAM_MODEL_O2,
    AS_SDK_CAM_MODEL_TANGGULA_B,
    AS_SDK_CAM_MODEL_BUTT
} AS_SDK_CAM_MODEL_E;
```

【参数】

Member	Description
AS_SDK_CAM_MODEL_UNKNOWN	未知类型
AS_SDK_CAM_MODEL_NUWA_XB40	XB40
AS_SDK_CAM_MODEL_NUWA_X100	X100
AS_SDK_CAM_MODEL_NUWA_HP60	HP60
AS_SDK_CAM_MODEL_NUWA_HP60V	HP60V
AS_SDK_CAM_MODEL_KUNLUN_A	KUNLUN-A
AS_SDK_CAM_MODEL_KUNLUN_C	KUNLUN-C
AS_SDK_CAM_MODEL_KONDYOR	KONDYOR(USB)
AS_SDK_CAM_MODEL_KONDYOR_NET	KONDYOR(NET)
AS_SDK_CAM_MODEL_HP60C	HP60C
AS_SDK_CAM_MODEL_HP60CN	HP60CN
AS_SDK_CAM_MODEL_VEGA	VEGA
AS_SDK_CAM_MODEL_CHANGA	CHANG-A
AS_SDK_CAM_MODEL_CHANGJIANG_B	CHANGJIANG-B
AS_SDK_CAM_MODEL_TANGGULA	TANGGULA
AS_SDK_CAM_MODEL_TANGGULA_A	TANGGULA-A
AS_SDK_CAM_MODEL_TAISHAN	TAISHAN
AS_SDK_CAM_MODEL_O2	O2

Member	Description
AS_SDK_CAM_MODEL_TANGGULA_B	TANGGULA-B

3.2.8 AS_CAM_ATTR_TYPE_E

【说明】

相机类型

【定义】

```
typedef enum AS_CAM_ATTR_TYPE {
    AS_CAMERA_ATTR_LNX_USB,
    AS_CAMERA_ATTR_WIN_USB,
    AS_CAMERA_ATTR_NET,
    AS_CAMERA_ATTR_BUTT
} AS_CAM_ATTR_TYPE_E;
```

【参数】

Member	Description
AS_CAMERA_ATTR_LNX_USB	Linux USB 类型
AS_CAMERA_ATTR_WIN_USB	Windows USB 类型
AS_CAMERA_ATTR_NET	网络类型
AS_CAMERA_ATTR_BUTT	无效值

3.2.9 AS_USB_DEV_ATTR_S

【说明】

Linux USB 类型结构体

【定义】

```
typedef struct AS_USB_DEV_ATTR {
    int vid;
    int pid;
    char serial[64];
    int bnum;
    int dnum;
    int port_number;
    char port_numbers[64];
} AS_USB_DEV_ATTR_S;
```

【参数】

Member	Description
vid	供应商识别码
pid	产品识别码
serial	SN 序列号
bnum	总线号

Member	Description
dnum	设备号
port_number	端口号
port_numbers	端口路径

3. 2. 10 AS_NETWORK_DEV_ATTR_S

【说明】

网络类型结构体

【定义】

```
typedef struct AS_NETWORK_DEV_ATTR {  
    char ip_addr[64];  
    int port;  
    AS_SDK_CAM_MODEL_E model;  
} AS_NETWORK_DEV_ATTR_S;
```

【参数】

Member	Description
ip_addr	ip 地址
port	端口号
model	相机型号

3. 2. 11 AS_USB_WIN_DEV_ATTR_S

【说明】

Windows USB 类型结构体

【定义】

```
#define WinBuffSize 2048  
typedef struct AS_USB_WIN_DEV_ATTR {  
    int vid;  
    int pid;  
    int deviceID;  
    char symbol_link[WinBuffSize];  
    char location_path[WinBuffSize];  
} AS_USB_WIN_DEV_ATTR_S;
```

【参数】

Member	Description
vid	供应商识别码
pid	产品识别码
deviceID	设备 ID
symbol_link	设备唯一标识码
location_path	设备路径

3. 2. 12 AS_CAM_ATTR_S

【说明】

相机信息结构体

【定义】

```
typedef struct AS_CAM_ATTR {
    AS_CAM_ATTR_TYPE_E type;
    union {
        AS_USB_DEV_ATTR_S usbAttrs;
        AS_NETWORK_DEV_ATTR_S netAttrs;
        AS_USB_WIN_DEV_ATTR_S winAttrs;
    } attr;
} AS_USB_WIN_DEV_ATTR_S;
```

【参数】

Member	Description
type	相机属性类型，指 union 中的属性类型
usbAttrs	Linux USB 信息结构体
netAttrs	网络信息结构体
winAttrs	Windows USB 信息结构体

3. 2. 13 AS_NET_DEV_Attrs_s

【说明】

网络设备信息，此信息支持供网络设备使用。

【定义】

```
typedef struct NET_DEV_ATTRS {
    char ip_addr[64];
    char mac_addr[64];
    int is_dhcp;
} AS_NET_DEV_Attrs_s;
```

【参数】

Member	Description
ip_addr	IP 地址
mac_addr	MAC 地址
is_dhcp	是否开启 DHCP。0 关闭，1 开启，-1 不设置

3. 2. 14 AS_TIME_STAMP_TYPE_E

【说明】

时间戳类型

【定义】

```
typedef enum TIME_STAMP_TYPE {
    AS_TIME_STAMP_TYPE_STEADY_CLOCK,
    AS_TIME_STAMP_TYPE_SYSTEM_CLOCK,
    AS_TIME_STAMP_TYPE_BUTT
} AS_TIME_STAMP_TYPE_E;
```

【参数】

Member	Description
AS_TIME_STAMP_TYPE_STEADY_CLOCK	系统单调时钟, 以开机时刻为 0 时刻开始计时
AS_TIME_STAMP_TYPE_SYSTEM_CLOCK	系统时间
AS_TIME_STAMP_TYPE_BUTT	无效值

4.ANG SDK 核心 API 说明

4.1 核心接口定义

4.1.1 初始化 SDK

[功能]

SDK 初始化

[格式]

```
int AS_SDK_Init();
```

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

在使用 SDK 的生命周期里，只需要初始化一次。多次初始化会返回成功，但是会打印警告信息。

4.1.2 去初始化 SDK

[功能]

SDK 去初始化

[格式]

```
int AS_SDK_Deinit();
```

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

与初始化接口 AS_SDK_Init() 配套使用，初始化到去初始化完成 SDK 的一个生命周期。

4.1.3 获取相机列表

[功能]

获取已连接的相机列表

[格式]

int AS_SDK_GetCameraList(std::list<AS_CAM_PTR> &devList);

[参数]

Type	Name	Description	IN/OUT
std::list<AS_CAM_PTR>	devList	相机列表	OUT

[返回值]

Type	Description
int	大于 0 ， 相机的数量； 其它值异常

[注意事项]

该接口所返回的获取相机列表，会同时记录在 SDK 内部。若需要重复调用，请先将已开流的相机关闭、已打开的相机关闭，然后将 AS_SDK_GetCameraList 得到的 devList 传入[释放相机列表](#)的接口将其释放掉，最后再次获取相机列表。否则将会出现未知错误。

4.1.4 释放相机列表

[功能]

释放相机列表资源

[格式]

int AS_SDK_FreeCameraList(std::list<AS_CAM_PTR> &devList);

[参数]

Type	Name	Description	IN/OUT
std::list<AS_CAM_PTR>	devList	相机列表	IN

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

调用后，devList 会被清空，里面的 camera 指针会被释放。之后再进行其他操作可能会出现未知错误。为防止内存泄露等未知错误，请在退出 SDK 前的适当位置调用该接口将相机列表资源释放。

4.1.5 打开相机

[功能]

打开指定相机

[格式]

int AS_SDK_OpenCamera(AS_CAM_PTR pCamera, const char *pParaFilePath);

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN
char*	pParaFilePath	相机配置文件路径	IN

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

相机配置文件在本文档上方 2.2.1 处说明。
重复调用之前需要先关闭相机，否则返回负值。

4.1.6 关闭相机

[功能]

关闭指定相机

[格式]

int AS_SDK_CloseCamera(AS_CAM_PTR pCamera);

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

4.1.7 注册图像数据回调函数

[功能]

注册图像数据回调函数

[格式]

int AS_SDK_RegisterStreamCallback(AS_CAM_PTR pCamera, AS_CAM_Stream_Cb_s *pstCallback);

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN
AS_CAM_Stream_Cb_s	pstCallback	回调函数	IN

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

特别注意，不允许在回调函数 pstCallback 里面调用 SDK 的接口，这很有可能产生死锁问题。同时，请不要在回调函数里执行阻塞/延时/耗时等操作，否则将产生未知问题。建议将回调上来的数据拷贝后使用，避免出现由于上一帧数据被下一帧数据覆盖，导致数据异常的问题。

4.1.8 注册图像融合数据回调函数

[功能]

注册图像融合数据回调函数

[格式]

int AS_SDK_RegisterMergeFrameCallback(AS_CAM_PTR pCamera, AS_CAM_Merge_Cb_s *pstCallback);

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN
AS_CAM_Merge_Cb_s	pstCallback	回调函数	IN

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

仅支持 kunlunA。特别注意，不允许在回调函数 pstCallback 里面调用 SDK 的接口，这很有可能产生死锁问题。同时，请不要在回调函数里执行阻塞/延时/耗时等操作，否则将产生未知问题。建议将回调上来的数据拷贝后使用，避免出现由于上一帧数据被下一帧数据覆盖，导致数据异常的问题。

4.1.9 打开相机数据流

[功能]

打开指定相机的图像

[格式]

```
int AS_SDK_StartStream(AS_CAM_PTR pCamera, int type);
```

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN
int	type	图像类型 IMG_TYPE_FLG，深度、点云等的一种或组合，需根据相机类型决定其有效参数。当为 0 时，SDK 将按照默认值输出。组合通过 操作，如 DEPTH_IMG_FLG POINT_CLOUD_FLG	IN

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

4.1.10 关闭相机数据流

[功能]

关闭指定相机的图像

[格式]

```
int AS_SDK_StopStream(AS_CAM_PTR pCamera, int type);
```

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN
int	type	图像类型 IMG_TYPE_FLG，深度、点云等的一种或组合，需根据相机类型决定其有效参数。当为 0 时，关闭全部图像	IN

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

该接口会阻塞直至图像数据回调函数执行结束。

4.1.11 启动设备监听功能

[功能]

启动设备监听功能，监听安思疆制造的相机插入或拔出的状态

[格式]

int AS_SDK_StartListener(const AS_LISTENER_CALLBACK_S &callback, AS_LISTENER_TYPE_E type, bool enumerate = true);

[参数]

Type	Name	Description	IN/OUT
AS_LISTENER_CALLBACK_S	callback	设备监听回调函数	IN
AS_LISTENER_TYPE_E	type	监听类型	IN
bool	enumerate	是否枚举设备	IN

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

此接口仅需调用一次，不可重复调用，否则将产生未知问题。若只使用 USB 相机，只需注册 USB 设备监听类型，反之只需注册网络监听类型。为降低 CPU 占用等情况，建议只注册需要的监听类型。

4.1.12 停止设备监听功能

[功能]

停止设备监听功能

[格式]

int AS_SDK_StopListener(void);

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

4.1.13 创建相机句柄

[功能]

创建相机句柄

[格式]

int AS_SDK_CreateCamHandle(AS_CAM_PTR &pCamera, AS_CAM_ATTR_S *attr);

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	OUT
AS_CAM_ATTR_S	attr	相机属性。	IN

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

若使用的是 [Linux-USB 相机](#)，输入的相机属性 attr 必须要有 bnum 和 dnum 的信息；若使用的是 [Win-USB 相机](#)，输入的相机属性 attr 必须要有 symbol_link 的信息；若使用的是[网络相机](#)，输入的相机属性 attr 必须要有 ip_addr 的信息。详细的相机属性详细参考 AS_CAM_ATTR_S 结构体。

4.1.14 销毁相机句柄

[功能]

销毁相机句柄

[格式]

int AS_SDK_DestroyCamHandle([AS_CAM_PTR](#) pCamera);

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

4.1.15 获取相机属性

[功能]

获取相机属性。

[格式]

int AS_SDK_GetCameraAttrs([AS_CAM_PTR](#) pCamera, [AS_CAM_ATTR_S](#) &attr);

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN
AS_CAM_ATTR_S	attr	相机属性	OUT

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

Linux-USB 的设别属性中，SN 号在此处暂不支持获取。SN 号的获取可通过 AS_SDK_GetSerialNumber()接口获取。对于非热插拔机制，在获取相机列表之后到释放相机列表之前的生命周期，均可调用此接口获取相机的属性；对于热插拔机制，在获得相机句柄之后到销毁句柄之前的生命周期，均可调用此接口获取相机的属性。

4.1.16 获取 SDK 版本号

[功能]

获取 sdk 版本号

[格式]

int AS_SDK_GetSwVersion(char *pszVersion, unsigned int size);

[参数]

Type	Name	Description	IN/OUT
char	pszVersion	sdk 版本号	OUT
unsigned int	size	pszVersion 空间最大值，单位字节	IN

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

无

4.2 通信接口定义

通信接口都是与相机固件进行通信，所以所有此类型的 API 请在调用 AS_SDK_OpenCamera() 打开相机之后才能使用，否则将返回错误。

4.2.1 获取相机固件版本号

[功能]

获取相机固件版本号

[格式]

```
int AS_SDK_GetFwVersion(AS_CAM_PTR pCamera, char *pszVersion, int size);
```

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN
char	pszVersion	相机固件版本号	OUT
int	size	pszVersion 空间最大值，单位字节	IN

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

无

4.2.2 获取流的类型

[功能]

获取流的图像类型

[格式]

```
int AS_SDK_GetStreamType(AS_CAM_PTR pCamera, int *type);
```

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN
int*	type	获取流图像类型(IMG_TYPE_FLG)	OUT

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

暂不支持。

4.2.3 获取相机 SN 序列号

[功能]

获取序列号

[格式]

```
int AS_SDK_GetSerialNumber(AS_CAM_PTR pCamera, char *pszSerialNo, int size);
```

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄	IN
char*	pszSerialNo	存储序列号的空间	OUT
int	size	pszSerialNo 空间最大值，单位字节	IN

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

在通过 AS_SDK_OpenCamera()打开指定的相机后,到 AS_SDK_CloseCamera()关闭相机的之前的整个生命周期均可调用此接口。

4.2.4 升级相机固件

[功能]

升级固件

[格式]

```
int AS_SDK_UpgradeDev(AS_CAM_PTR pCamera, char *pszFilePath, AS_CAM_Upgrade_Development *pstCallback);
```

[固件包格式]

└─xxx.zip	总压缩包
└─xx_a.bin	模组应用
└─xx_k.bin	模组内核
└─xx_b.bin	背景图
└─xx_h.bin	Himax 文件
└─xx_c.bin	参考图/标定图
└─xx_cb.bin	备用参考图/标定图
└─xx_p.bin	内外参文件
└─xx_x.bin	MAPX 文件
└─xx_y.bin	MAPY 文件
└─xx_r.bin	ISP 文件
└─xx_wa.bin	Wrap 文件
└─xx_wb.bin	备用 Wrap 文件

└─xx_waxy.bin
└─xx_wbxy.bin

Wrap_xy 文件
备用 Wrap_xy 文件

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN
char*	pszFilePath	待升级的固件路径	IN
AS_CAM_Upgrade_Dev_s	pstCallback	升级状态回调函数	IN

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

不支持在开流时升级固件，如果已经开流，请先关流再升级。升级工具可找 FAE 获取。

4.2.5 获取相机内外参

[功能]

获取指定相机的内外参

[格式]

int AS_SDK_GetCamParameter(AS_CAM_PTR pCamera, AS_CAM_Parameter_s *pstParam);

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN
AS_CAM_Parameter_s	pstParam	相机内外参	OUT

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

对于 HP60C/HP60CN/VAGE 型号的模组，需要调用 AS_SDK_StartStream 开启流并获取到图像数据之后，才能获取到正确的内外参。其余型号的模组在开相机后即可获取。

4.2.6 获取相机固件的模式

[功能]

获取相机固件的模式

[格式]

```
int AS_SDK_GetTofMode(AS_CAM_PTR pCamera, AS_TOFMODE_E &mode);
```

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN
AS_TOFMODE_E	mode	固件模式	OUT

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

仅支持 KUNLUN 系列的相机。

4.2.7 获取分辨率能力集

[功能]

获取分辨率能力集

[格式]

```
int AS_SDK_GetCapability(AS_CAM_PTR pCamera, AS_MEDIA_TYPE_E type, std::vector<AS_STREAM_Param_s> &capability);
```

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN
AS_MEDIA_TYPE_E	type	图像类型	IN
std::vector<AS_STREAM_Param_s>	capability	相机图像参数	IN

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

用户在开相机之后，可通过此接口获取能力集，根据需求通过 [设置图像参数](#) 接口设置。对于

4.2.8 设置图像参数

[功能]
设置图像参数

[格式]
int AS_SDK_SetStreamParam(AS_CAM_PTR pCamera, AS_MEDIA_TYPE_E type,AS_STREAM_Param_s *pstStreamParam);

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN
AS_MEDIA_TYPE_E	type	流类型	IN
AS_STREAM_Param_s	pstStreamParam	相机图像参数	IN

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]
可通过 AS_SDK_GetCapability()接口去获取可支持的分辨率能力集。对于 HP60C/HP60CN/VAGE 的模组，要求 RGB 和 DEPTH/IR（如有 IR）设置的帧率一致。所有型号的模组，设置的参数只支持从获取分辨率能力集接口获取的帧率和分辨率。特别注意，帧率和分辨率必须在开流前设置才能生效。

4.2.9 获取图像参数

[功能]
获取图像参数

[格式]
int AS_SDK_GetStreamParam(AS_CAM_PTR pCamera, AS_STREAM_Param_s *pstStreamParam);

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN
AS_STREAM_Param_s	pstStreamParam	相机图像参数	OUT

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

4.2.10 获取相机型号

[功能]

获取相机型号

[格式]

```
int AS_SDK_GetCameraModel(AS_CAM_PTR pCamera, AS_SDK_CAM_MODEL_E &model);
```

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN
AS_SDK_CAM_MODEL_E	model	相机型号	OUT

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

在获取相机列表之后即可调用。

4.2.11 设置相机网络信息

[功能]

设置相机网络信息

[格式]

```
int AS_SDK_SetDevNetAttrs(AS_CAM_PTR pCamera, AS_NET_DEV_Attrs_s attrs);
```

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN
AS_NET_DEV_Attrs_s	attrs	包括 ip 地址、mac 地址、是否开启 dhcp	IN

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

注意网络信息属性输入要求，详见 AS_NET_DEV_Attrs_s。

4. 2. 12 获取相机网络信息

[功能]

获取相机网络信息

[格式]

```
int AS_SDK_GetDevNetAttrs(AS_CAM_PTR pCamera, AS_NET_DEV_Attrs_s &attrs);
```

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN
AS_NET_DEV_Attrs_s	attrs	包括 ip 地址、mac 地址、是否开启 dhcp	OUT

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

无。

4. 2. 13 设置时间戳类型

[功能]

设置时间戳类型

[格式]

```
int AS_SDK_SetTimeStampType(AS_CAM_PTR pCamera, AS_TIME_STAMP_TYPE_E type);
```

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN
AS_TIME_STAMP_TYPE_E	type	时间戳类型	IN

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

仅支持 linux 版本 hp60c。

4. 2. 14 获取相机配置信息

[功能]

获取相机配置信息

[格式]

```
int AS_SDK_GetConfigInfo(AS_CAM_PTR pCamera, AS_CONFIG_INFO_S &config_info);
```

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN
AS_CONFIG_INFO_S	config_info	配置信息	OUT

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

仅支持 RGBD 相机使用，此接口为了获取 RGB 和深度是否对齐，以及点云有序/无序的状态。

4.2.15 获取相机私有数据的空间大小

[功能]

获取相机私有数据的空间大小

[格式]

```
int AS_SDK_GetPrivateMaxSize(AS\_CAM\_PTR pCamera, int &size);
```

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN
int	size	获取到的私有数据的空间大小	OUT

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

仅支持 KUNLUN-A 和 KUNLUN-C 相机

4.2.16 向相机写入私有数据

[功能]

向相机写入私有数据

[格式]

```
int AS_SDK_WritePrivateData(AS\_CAM\_PTR pCamera, void *data, int size, int offset);
```

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN
void	data	私有数据	IN
int	size	私有数据大小	IN
int	offset	相机内私有数据空间的偏移量	IN

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

仅支持 KUNLUN-A 和 KUNLUN-C 相机

4.2.17 从相机读取私有数据

[功能]

从相机读取私有数据

[格式]

```
int AS_SDK_ReadPrivateData(AS_CAM_PTR pCamera, void *data, int size, int offset);
```

[参数]

Type	Name	Description	IN/OUT
AS_CAM_PTR	pCamera	相机句柄，由 AS_SDK_GetCameraList 接口或 AS_CreateCamHandle 接口得到相机句柄。	IN
void	data	私有数据	OUT
int	size	私有数据大小	IN
int	offset	相机内私有数据空间的偏移量	IN

[返回值]

Type	Description
int	0 成功 其他值异常

[注意事项]

仅支持 KUNLUN-A 和 KUNLUN-C 相机

5. 帮助

5.1 注意事项

此 SDK 提供的 ROS 包无需移动到 ROS 的工作空间中，直接根据 2.2 小节提供的方法编译运行即可。

5.2 FAQ

5.2.1 如何不以 root 权限运行 Linux 程序

在 Linux 系统中，访问设备需要 root 权限，对于通过操作/dev 目录下的设备节点方式来操作设备的程序，一种方式是通过 chmod 命令将设备节点的权限进行修改，但这仅是临时有效，而且对于非通过/dev 设备节点操作设备的程序，将无法通过 chmod 更改权限。此时，可通过 Linux 的 udev 和 rules 规则来管理设备的权限。

对于使用安思疆相机模组的 Linux 系统，可以通过执行我们的 SDK 包 demo/linux_ros/linux/scripts 目录中 create_udev_rules.sh 脚本，即可通过普通权限运行 linux 程序访问安思疆相机。

5.2.2 如何不以 root 权限运行 ROS 节点

在 Linux 系统中，访问设备需要 root 权限，对于通过操作/dev 目录下的设备节点方式来操作设备的程序，一种方式是通过 chmod 命令将设备节点的权限进行修改，但这仅是临时有效，而且对于非通过/dev 设备节点操作设备的程序，将无法通过 chmod 更改权限。此时，可通过 Linux 的 udev 和 rules 规则来管理设备的权限。

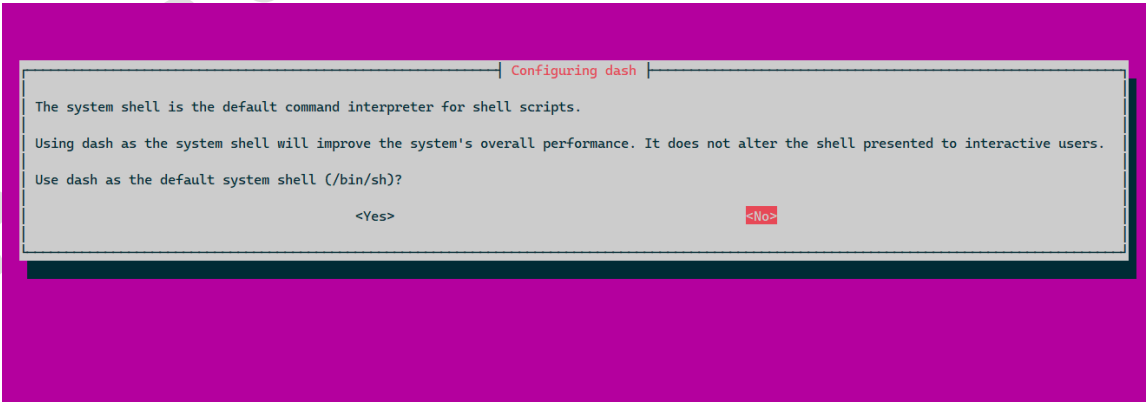
对于使用安思疆相机模组的 Linux 系统，可通过执行 demo/linux_ros/ros/src/ascamera/scripts 目录中 create_udev_rules.sh 脚本，即可通过普通权限运行 ROS 节点访问安思疆相机。修改 demo/ros/ 目录中 run_ascamera_node.sh 脚本，将检查并申请 root 权限的语句注释即可。如下图

```
18
19 CUR_DIR="$(dirname "$(realpath "${BASH_SOURCE[0]}")")"
20 # check for whitespace in $CUR_DIR and exit for safety reasons
21 grep -q "[[:space:]]" <<<"${CUR_DIR}" && { echo "\"${CUR_DIR}\" contains whitespace. Not supported. Aborting." >&2 ; exit 1 ; }
22
23 # if [ $EUID -ne 0 ]; then
24 # ... echo -e "${RED}---This script requires root privileges, trying to use sudo${NORMAL}"
25 # ... sudo "${CUR_DIR}/run_ascamera_node.sh" "$@"
26 # ... exit $?
27 # fi
28
29 if [ -f /opt/ros/melodic/setup.bash ]; then
30 ... source /opt/ros/melodic/setup.bash
31 elif [ -f /opt/ros/kinetic/setup.bash ]; then
32 ... source /opt/ros/kinetic/setup.bash
33 elif [ -f /opt/ros/noetic/setup.bash ]; then
34 ... source /opt/ros/noetic/setup.bash
35 else
36 ... echo "Error, Can't not found ros in /opt/"
37 fi
38
39 if [ "$(ps -aux | grep rosmaster | grep -v grep | wc -l)" -eq "0" ]; then
40 ... roscore &
41 ... sleep 3
42 fi
```

5.2.3 执行 SDK 提供的脚本报错

执行 SDK 提供的脚本报:Bad substitution 或 Syntax error: redirection unexpected 等错误。这是由于 SDK 提供的 shell 脚本为 bash shell，而 ubuntu 系统的默认 shell 解析器为 dash。Bash 比 Dash 更强大，部分 Bash 语法可能不被 dash 解析。可通过以下步骤将 ubuntu 的默认 shell 更改为 bash。

在终端执行 sudo dpkg-reconfigure dash ，在弹出的窗口选择 NO。



5.2.4 在 ubuntu20.04/ubuntu22.04 ROS 上编译报错

在 ubuntu20.04/ubuntu22.04 的 ROS（Noetic 或更高版本）编译报以下错误。

```

    from /opt/ros/noetic/include/ros/time.h:58,
    from /opt/ros/noetic/include/ros/ros.h:38,
    from /home/boocax/roswork/src/as_nuwacam/src/as_nuwacam_node.cpp:29:
/usr/include/boost/mpl/aux_/preprocessed/gcc/minus.hpp:68:8: note:   'boost::mpl::minus'
68 | struct minus
    | ^~~~~
In file included from /usr/include/pcl-1.10/pcl/point_types.h:44,
    from /usr/include/pcl-1.10/pcl/common/impl/copy_point.hpp:41,
    from /usr/include/pcl-1.10/pcl/common/copy_point.h:58,
    from /usr/include/pcl-1.10/pcl/common/impl/io.hpp:45,
    from /usr/include/pcl-1.10/pcl/common/io.h:586,
    from /usr/include/pcl-1.10/pcl/io/file_io.h:41,
    from /usr/include/pcl-1.10/pcl/io/pcd_io.h:44,
    from /opt/ros/noetic/include/pcl_conversions/pcl_conversions.h:70,
    from /home/boocax/roswork/src/as_nuwacam/src/as_nuwacam_node.cpp:38:
/usr/include/pcl-1.10/pcl/point_types.h:698:1: error: 'minus' is not a member of 'pcl::traits'
698 | POINT_CLOUD_REGISTER_POINT_STRUCT (pcl::PointDEM,
    | ^~~~~~
/usr/include/pcl-1.10/pcl/point_types.h:698:1: note: suggested alternatives:
In file included from /usr/include/c++/9/string:48,
    from /usr/include/c++/9/bits/locale_classes.h:40,
    from /usr/include/c++/9/bits/ios_base.h:41,
    from /usr/include/c++/9/ios:42,
    from /usr/include/c++/9/ostream:38,
    from /usr/include/c++/9/iostream:39,
    from /home/boocax/roswork/src/as_nuwacam/src/as_nuwacam_node.cpp:21:
/usr/include/c++/9/bits/stl_function.h:177:12: note:   'std::minus'
177 | struct minus : public binary_function<_Tp, _Tp, _Tp>
    | ^~~~~

```

由于我们的 ROS 下的节点示例程序是基于 ubuntu18.04 上的 melodic 版本的 ROS 进行开发，当移植到 ROS-Noetic 时，可将 ros/src/ascamera/CMakeLists.txt 文件中的 -std=c++11 更改为 -std=c++14 或 -std=c++17

sample > ros > src > ascamera > CMakeLists.txt

```

1  cmake_minimum_required(VERSION 3.0.2)
2  project(ascamera)
3
4  ## Compile as C++11, supported in ROS Kinetic and newer
5  add_compile_options(-std=c++11)
6  add_definitions(-std=c++11)
7  # get gcc -v target

```

5.2.5 NUWA 系列相机在虚拟机中匹配异常问题

由于 NUWA 系列的相机是由 2 个 usb 设备组合为 1 个相机（1 个 usb 通信设备+1 个 uvc 设备），所以需要进行匹配；当在虚拟机上运行时，由于虚拟机的设备拓扑结构可能不正确导致 usb 设备无法进行正确配对，所以当使用虚拟机运行程序的时候，只能接一个 NUWA 系列的相机，超过一个相机就有可能因上述原因导致匹配失败。

5.2.6 KUNLUN-A 相机在虚拟机中运行异常

KUNLUN-A 型号的模组暂时只支持物理机，无法在虚拟机上使用。

5.2.7 多相机运行出流异常问题

导致此问题可能之一是系统内核参数 `usbfs_memory_mb` 设置太小。`usbfs_memory_mb` 是一个内核参数，用于指定 USB 文件系统（`usbfs`）使用的内存大小。`usbfs` 是一种虚拟文件系统，用于在用户空间和内核之间传输 USB 设备数据。该参数指定了 `usbfs` 分配给 USB 设备通信缓冲区的内存大小，以 MB 为单位。默认情况下，这个值是 16 MB。通过增加或减少该值，你可以调整 USB 设备通信缓冲区的内存大小。

临时修改方法(系统重启后会失效)：把此内核参数改大至 64/128/512 或更大。

命令：`echo "64" | sudo tee -a /sys/module/usbcore/parameters/usbfs_memory_mb`

永久修改方法客户可根据所使用的系统自行设置。

5.2.8 在 windows10/windows11 打开 USB 相机失败

若在 windows 上打开相机失败，且 sdk 日志打印 "SETUP -COM already setup - threaded VI might not be possible. 导致此问题的原因是因为 windows 上视频流捕获 sdk 需要用到 sdk 组件，而 sdk 设计是用 com 的多线程模式，但是某些软件例如 QT 的组件也使用了 com 组件，但是是单示例模式，模式不一样的情况可能导致 sdk 相机打开失败。

因为大部分需要禁用 COM 组件使用，或者使用 sdk 前使用 `CoUninitialize(void)` 反初始化 Com 组件，然后正常启动 sdk，由 sdk 决定 com 组件的模式即可。

服务及售后支持

邮箱: info@angstrong.com

sales@angstrong.com

网站: www.angstrong.com

电话: 0755-86568667

本文档如有更新，恕不另行通知。

如果您对文档有任何疑问或建议，请通过电子邮箱联系我们：

info@angstrong.com

Copyright 2018-2024 安思疆版权所有

